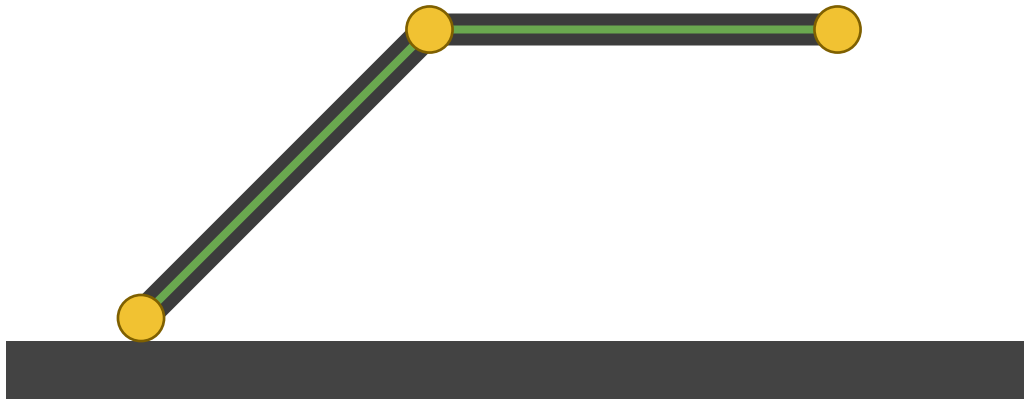
Code Implementation of Forward Kinematics and Other Methods

Robotic Arm Course · Lesson 5

Day 1: The flat arm in Python
Day 2: 3D, two methods, and the arm’s reach



Where Lesson 4 left off:

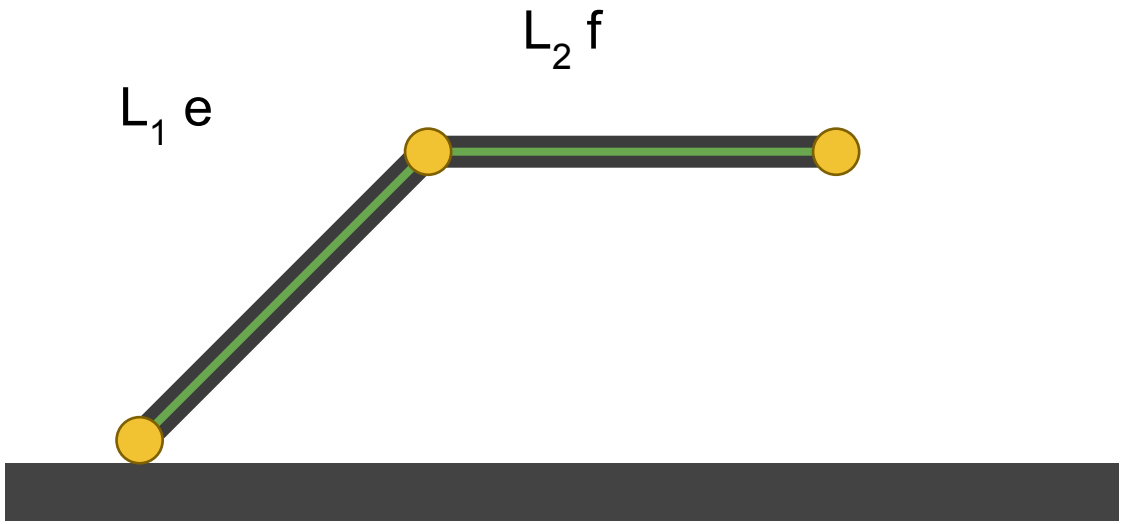
A link's arrow: $(\cos \theta, \sin \theta)$

Turn 90°: $(x, y) \rightarrow (-y, x)$

The forearm's arrow:
$f = \cos \theta_E \cdot e + \sin \theta_E \cdot n$

The tip: $L_1\, e + L_2\, f$

Today Python does these steps.



Setting up:

1. Open Python (your usual editor).
2. Open arm_starter.py.
3. Save a copy as arm.py.
4. Run it. Nothing prints yet. If there is no error, it works.

```
import math

L1 = 250   # upper arm (mm)
L2 = 250   # forearm (mm)

def show(v):
    print([round(n, 2) + 0.0 for n in v])
```

import math gives us math.sin and more.
L1 and L2 are the link lengths.
show() prints a vector rounded to 2 decimals.

We add to this file, one function at a time.

Python’s sin and cos use radians:

Angles in Lesson 4 were in degrees.
math.sin and math.cos want radians.
math.radians() changes degrees into radians.

```
import math

print(math.radians(180))                # 3.141592653589793
print(math.sin(math.radians(30)))       # 0.4999999999999994
print(round(math.sin(math.radians(30)), 2))  # 0.5
```

round(x, 2) keeps 2 decimals.
Python shows tiny errors like 0.4999999999999994. That is normal.

Try it yourself #1: what does each line print?

```
print(round(math.sin(math.radians(30)), 2))
print(round(250 * math.cos(math.radians(45)), 2))
print(math.sqrt(3**2 + 4**2))
```

Guess first. Then run them to check.

Hint: line 2 is a number from Lesson 4.
Line 3 is the length of the vector (3, 4).

Try it yourself #1: solution

```
print(round(math.sin(math.radians(30)), 2))
print(round(250 * math.cos(math.radians(45)), 2))
print(math.sqrt(3**2 + 4**2))
```

```
0.5
176.78
5.0
```

176.78 is the upper arm's number from Lesson 4.
5.0 is the length of (3, 4). Python shows 5.0, not 5.

Step 1: a function for one arrow:

A function is a recipe with a name.
Numbers go in. One answer comes out.



Input: the angle theta, in degrees, measured from the floor.
Output: the link's arrow [cos θ, sin θ]. Its length is 1.

Step 1: arrow

```
def arrow(theta):
    theta = math.radians(theta)
    return [math.cos(theta), math.sin(theta)]
```

Test it:

```
show(arrow(45))
show(arrow(90))
```

```
[0.71, 0.71]
[0.0, 1.0]
```

def starts the function.

The first line changes degrees to radians.

return gives back [cos θ, sin θ] as a list.

arrow(45) is e, the upper arm's arrow from Lesson 4. show rounds 0.7071 to 0.71.

Step 2: multiplying and adding vectors:

To multiply a vector by a number, multiply each number:
250 × (0.7071, 0.7071) = (176.78, 176.78)
To add two vectors, add the numbers in the same position:
(3, 4) + (2, −1) = (5, 3)

i	**u[i]**	**v[i]**	**u[i] + v[i]**
0	3	2	5
1	4	−1	3

i is the position of a number in the list.
Python starts counting at 0.

A loop can go through every position.

Step 2: scale and add_vectors

```
def scale(k, v):
    result = []
    for i in range(len(v)):
        result.append(k * v[i])
    return result

def add_vectors(u, v):
    result = []
    for i in range(len(u)):
        result.append(u[i] + v[i])
    return result
```

Test it:

```
show(scale(250, arrow(45)))
print(add_vectors([3, 4], [2, -1]))
```

```
[176.78, 176.78]
[5, 3]
```

result = [] starts an empty list.

range(len(v)) counts 0, 1, ... once for each number in v.

append adds the new number to the list.

return is lined up with for. It is not inside the loop.

scale(250, arrow(45)) is the upper arm vector from Lesson 4.

Try it yourself #2: find the link vector

Use scale and arrow to print the link vector for:

1. L=250mm, θ=30°
2. L=250mm, θ=90°

Compare with Lesson 4, Try it yourself #1.

Try it yourself #2: solution

Use scale and arrow to print the link vector for:

1. L=250mm, θ=30°
2. L=250mm, θ=90°

Compare with Lesson 4, Try it yourself #1.

```
show(scale(250, arrow(30)))
show(scale(250, arrow(90)))
```

```
[216.51, 125.0]
[0.0, 250.0]
```

Python shows 125.0, not 125.00.
It is the same number.

Step 3: turning arrows

```
def turn_90(v):
    return [-v[1], v[0]]

def turn(e, n, theta):
    theta = math.radians(theta)
    a = scale(math.cos(theta), e)
    b = scale(math.sin(theta), n)
    return add_vectors(a, b)
```

Test it:

```
show(turn_90([3, 4]))
show(turn([1, 0], [0, 1], 30))
```

```
[-4.0, 3.0]
[0.87, 0.5]
```

turn_90 is the rule from Lesson 4:
$(x, y) \rightarrow (-y, x)$

turn is the forearm rule:
$\cos \theta \cdot e + \sin \theta \cdot n$

With e = [1, 0] and n = [0, 1], turn gives the unit circle: [cos 30°, sin 30°].

Step 4: the flat arm

```
def flat_tip(theta_s, theta_e):
    e = arrow(theta_s)
    n = turn_90(e)
    f = turn(e, n, theta_e)
    return add_vectors(scale(L1, e), scale(L2, f))
```

Each line is one row of the link table from Lesson 4.

e is the upper arm's arrow, at θ_S from the floor.

n is e turned 90°.

f is the forearm's arrow: e turned by θ_E.

The tip is L_1 e + L_2 f.

Run the worked example:

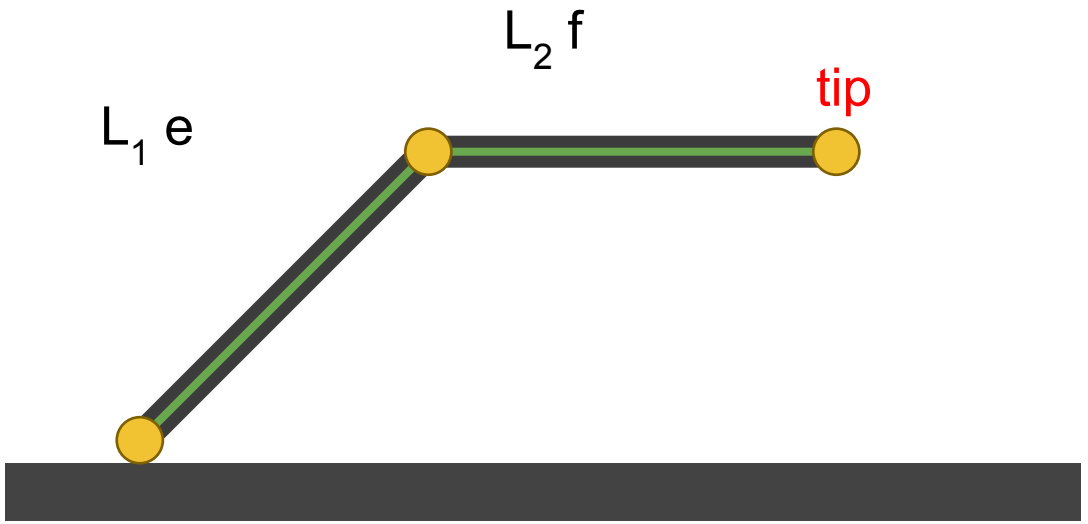
```
show(flat_tip(45, -45))
```

```
[426.78, 176.78]
```

The same answer as Lesson 3 and Lesson 4.

$\theta_S = 45°$ and $\theta_E = -45°$ (the elbow bends down)

Inside the function:
e = [0.71, 0.71]
n = [-0.71, 0.71]
f = [1.0, 0.0]
L_1 e + L_2 f = [426.78, 176.78]



Try it yourself #3: run flat_tip

Use flat_tip to find the tip for:

1. θ_S=60° and θ_E=−30°
2. θ_S=30° and θ_E=0°

Then compare with your link tables from Lesson 4 (Try it yourself #2 and #3).

Try it yourself #3: solution

Use flat_tip to find the tip for:

1. $\theta_S=60°$ and $\theta_E=-30°$
2. $\theta_S=30°$ and $\theta_E=0°$

Then compare with your link tables from Lesson 4 (Try it yourself #2 and #3).

```
show(flat_tip(60, -30))
show(flat_tip(30, 0))
```

```
[341.51, 341.51]
[433.01, 250.0]
```

The same answers as in Lesson 4.

Bug hunt: what went wrong?

We test with θ_S=45° and θ_E=−45°. The right answer is [426.78, 176.78].

A

```
def arrow(theta):
    return [math.cos(theta),
            math.sin(theta)]
```

prints:

```
[374.62, 270.28]
```

B

```
def turn_90(v):
    return [v[1], -v[0]]
```

prints:

```
[176.78, 426.78]
```

C

```
def add_vectors(u, v):
    result = []
    for i in range(len(u)):
        result.append(u[i] + v[i])
        return result
```

prints:

```
[426.78]
```

Find the mistake in each one.

Bug hunt: answers

We test with θ_S=45° and θ_E=−45°. The right answer is [426.78, 176.78].

A

```
def arrow(theta):
    return [math.cos(theta),
            math.sin(theta)]
```

prints:

```
[374.62, 270.28]
```

math.radians is missing. Python reads 45 as 45 radians.

B

```
def turn_90(v):
    return [v[1], -v[0]]
```

prints:

```
[176.78, 426.78]
```

This turns the arrow clockwise. The rule is (x, y) → (−y, x).

C

```
def add_vectors(u, v):
    result = []
    for i in range(len(u)):
        result.append(u[i] + v[i])
        return result
```

prints:

```
[426.78]
```

return is inside the loop. It stops after the first number.

Check new code against an answer you already know, like Lesson 4.

Day 1 recap:

arrow: a link's arrow (length 1)

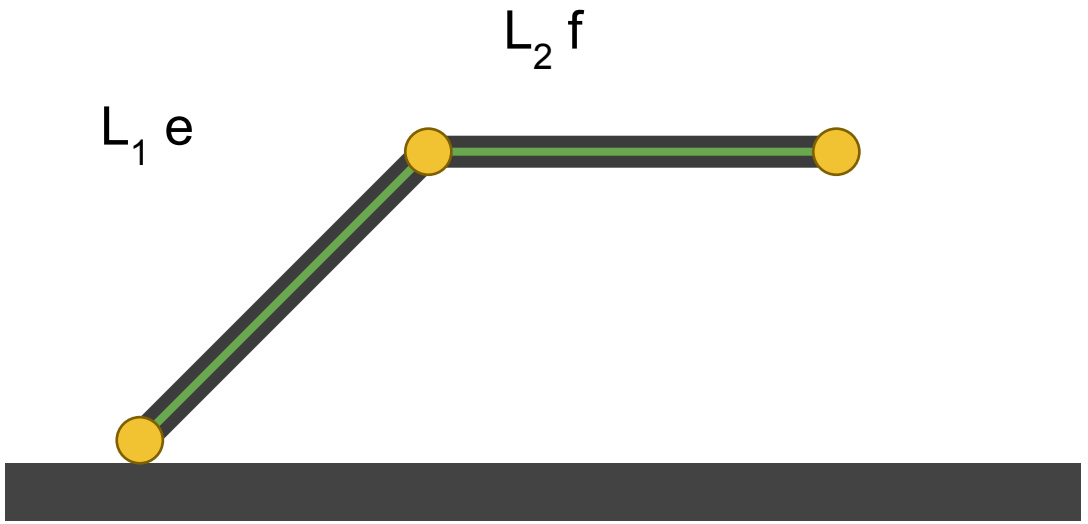
scale and add_vectors: multiply and add vectors

turn_90 and turn: turn an arrow

flat_tip: the tip of the flat arm

When the answer is wrong, check it against an answer you already know.

Tomorrow the base turns, so each vector gets a third number.



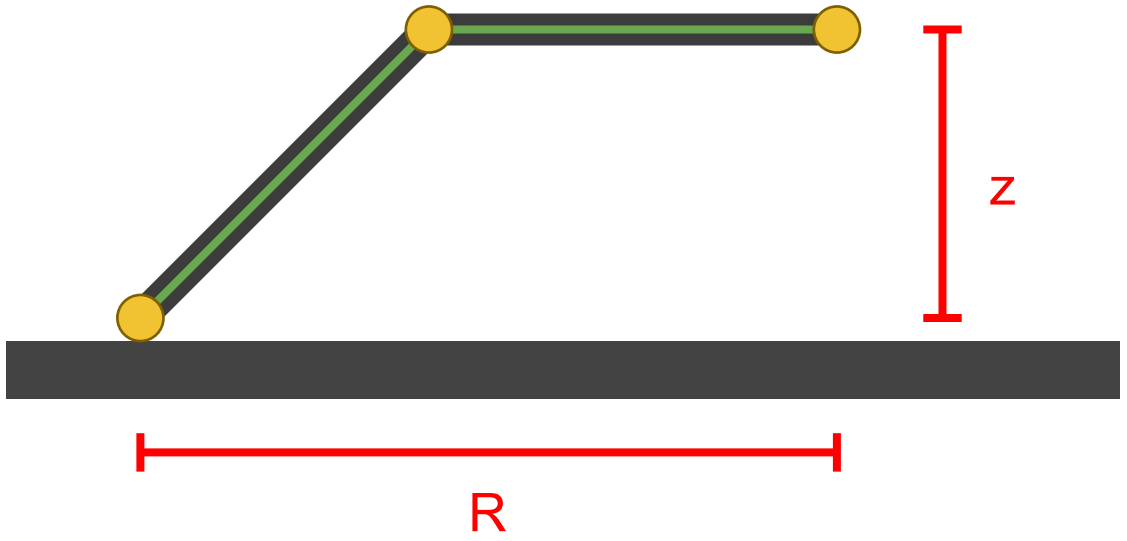
Day 2: where Day 1 left off

1. What does arrow(90) return?

2. What does turn_90([0.6, 0.8]) return?

3. What does flat_tip(45, -45) return?

Today the base turns.
Each vector gets a third number: (x, y, z).



Step 5: the arrows in 3D

```
def arrow_3d(theta_b, theta_s):
    b = math.radians(theta_b)
    s = math.radians(theta_s)
    x = math.cos(s) * math.cos(b)
    y = math.cos(s) * math.sin(b)
    z = math.sin(s)
    return [x, y, z]

def turned_3d(theta_b, theta_s):
    return arrow_3d(theta_b, theta_s + 90)
```

arrow_3d is e. turned_3d is n.

The formula from Lesson 4:

$e = (\cos \theta_S \cos \theta_B, \cos \theta_S \sin \theta_B, \sin \theta_S)$

Lesson 4's check: n is e with θ_S + 90° in place of θ_S.

So turned_3d just calls arrow_3d.

Test the 3D arrows:

```
show(arrow_3d(45, 45))
show(turned_3d(45, 45))
show(scale(250, arrow_3d(45, 45)))
```

```
[0.5, 0.5, 0.71]
[-0.5, -0.5, 0.71]
[125.0, 125.0, 176.78]
```

These are e, n and the upper arm vector from the worked example in Lesson 4, with θ_B = 45° and θ_S = 45°.

Lesson 4's table shows 0.7071. show rounds it to 0.71.

Step 6: the whole arm in 3D

```
def tip_3d(theta_b, theta_s, theta_e):
    e = arrow_3d(theta_b, theta_s)
    n = turned_3d(theta_b, theta_s)
    f = turn(e, n, theta_e)
    return add_vectors(scale(L1, e), scale(L2, f))
```

Compare it with flat_tip: only the lines for e and n changed.

turn, scale and add_vectors did not change. Their loops work for three numbers just as well as two.

This is the DH idea: the forearm only needs the upper arm’s arrows, e and n.

Run the worked example:

```
show(tip_3d(45, 45, -45))
```

```
[301.78, 301.78, 176.78]
```

$\theta_B = 45°$, $\theta_S = 45°$, $\theta_E = -45°$

The same answer as Lesson 4.

https://www.desmos.com/3d/qq1w1zrs78

Check it in Desmos:

1. Open the Desmos graph from Lesson 4.
2. Set u = 45, v = 45, w = −45.
3. d should read (3.0178, 3.0178, 1.7678)
4. Multiply by 100: (301.78, 301.78, 176.78)

$u = \theta_B$, $v = \theta_S$, $w = \theta_E$

Try it yourself #4: run tip_3d

Find the tip for:

1. $\theta_B=30°$, $\theta_S=30°$, $\theta_E=0°$
2. $\theta_B=30°$, $\theta_S=60°$, $\theta_E=-90°$

These are Lesson 4, Try it yourself #4 and #5. Check in Desmos too.

Try it yourself #4: solution

Find the tip for:

1. $\theta_B=30°$, $\theta_S=30°$, $\theta_E=0°$
2. $\theta_B=30°$, $\theta_S=60°$, $\theta_E=-90°$

These are Lesson 4, Try it yourself #4 and #5. Check in Desmos too.

```
show(tip_3d(30, 30, 0))
show(tip_3d(30, 60, -90))
```

```
[375.0, 216.51, 250.0]
[295.75, 170.75, 91.51]
```

Another method: Lesson 3 in code

```
def trig_tip(theta1, theta2, theta3):
    a1 = math.radians(theta1)
    a2 = math.radians(theta1 + theta2)
    b = math.radians(theta3)
    r = L1 * math.sin(a1) + L2 * math.sin(a2)
    z = L1 * math.cos(a1) + L2 * math.cos(a2)
    x = r * math.cos(b)
    y = r * math.sin(b)
    return [x, y, z]
```

This uses Lesson 3's angles and triangles. θ_1 is measured from straight up.

It is a different method for the same arm.

Lesson 3's steps:

$r = L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2)$

$z = L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2)$

$x = r \cos \theta_3$

$y = r \sin \theta_3$

r is the reach. z is the height (Lesson 3 called it Y).

Do the two methods agree?

```
poses = [(45, 45, 45), (60, 0, 30), (30, 90, 30)]

for pose in poses:
    theta1, theta2, theta3 = pose
    print(pose)
    show(trig_tip(theta1, theta2, theta3))
    show(tip_3d(theta3, 90 - theta1, -theta2))
```

prints:

```
(45, 45, 45)
[301.78, 301.78, 176.78]
[301.78, 301.78, 176.78]
(60, 0, 30)
[375.0, 216.51, 250.0]
[375.0, 216.51, 250.0]
(30, 90, 30)
[295.75, 170.75, 91.51]
[295.75, 170.75, 91.51]
```

The poses use Lesson 3's angles.
tip_3d needs DH angles, so we use Lesson 4's rule:
$\theta_B = \theta_3,\ \theta_S = 90° - \theta_1,\ \theta_E = -\theta_2$

The two answers match every time. If they ever differ, one of them has a bug.

Where can the tip go?

A function can be used again and again.
A loop can try every pose.

We let θ_S go from 0° to 90° and θ_E go from −90° to 0°, in steps of 10°.
We keep $\theta_B = 0°$, so we look at the arm from the side.

```
for theta_e in range(-90, 1, 10):
    print(theta_e)
```

This prints −90, −80 ... 0.
range(-90, 1, 10) stops before 1.

Sweeping the angles:

```
distances = []
for theta_s in range(0, 91, 10):
    for theta_e in range(-90, 1, 10):
        x, z = flat_tip(theta_s, theta_e)
        distances.append(math.sqrt(x ** 2 + z ** 2))

print(len(distances), "poses")
print("farthest:", round(max(distances), 2), "mm")
print("nearest:", round(min(distances), 2), "mm")
```

```
100 poses
farthest: 500.0 mm
nearest: 353.55 mm
```

One loop inside another tries every pair of angles.

x, z = ... takes the two numbers out of the list.

math.sqrt(x ** 2 + z ** 2) is the distance from the base (Pythagorean theorem).

Try it yourself #5: change the elbow range

In reach_sweep.py, change the elbow loop to:

```
    for theta_e in range(-150, 1, 10):
```

Will the nearest distance get bigger or smaller?

Guess first. Then run it.

Try it yourself #5: solution

In reach_sweep.py, change the elbow loop to:

```
    for theta_e in range(-150, 1, 10):
```

```
160 poses
farthest: 500.0 mm
nearest: 129.41 mm
```

A bent elbow brings the tip closer to the base.

For example, $\theta_S = 0°$ and $\theta_E = -150°$ gives $2 \times 250 \times \cos 75° = 129.41$ mm.

What we learned:

A function is a recipe: numbers in, one answer out.

Each DH step from Lesson 4 became a small function:
arrow, scale, add_vectors, turn_90, turn, flat_tip, arrow_3d, turned_3d, tip_3d

tip_3d used turn, scale and add_vectors without any changes.

When the answer is wrong, check it against an answer you already know.

Two methods gave the same answer.

Loops can try many poses to show the arm’s reach.

Next lesson: inverse kinematics. We start from the target and find the angles.